

Date: 31-8-26**Lab # 02 TO IMPLEMENT CONDITIONAL STATEMENTS AND LOOPS IN PYTHON**

Objective:

- Understand and apply basic control structures such as if, else, while, and for.
- Learn to use nested loops and nested conditions.
- Practice writing efficient programs using these control structures.

Lab Theory: Control Structures

Control structures are essential elements in programming that allow developers to dictate the flow of execution within a program. These structures enable the program to make decisions, repeat tasks, or perform complex logic based on certain conditions.

1. Conditional Statements: if, else, elif

Conditional statements allow a program to take different paths based on certain conditions. In Python (or other similar languages), these statements include if, else, and elif.

1.1 if Statement:

The if statement evaluates a condition and, if it is true, executes a block of code.

Syntax:

```
python
Copy code
if condition:
    # Code to execute if the condition is True
```

Example:

```
python
Copy code
x = 5
if x > 0:
    print("x is positive")
```

1.2 else Statement:

The else statement executes a block of code if the if condition is False.

Syntax:

```
python
Copy code
```

```
if condition:
    # Code if condition is True
else:
    # Code if condition is False
```

Example:

```
python
Copy code
x = -3
if x > 0:
    print("x is positive")
else:
    print("x is negative or zero")
```

1.3 elif Statement:

The elif (short for "else if") statement checks multiple conditions, one after the other.

Syntax:

```
python
Copy code
if condition1:
    # Code if condition1 is True
elif condition2:
    # Code if condition1 is False and condition2 is True
else:
    # Code if all previous conditions are False
```

Example:

```
python
Copy code
x = 10
if x > 10:
    print("x is greater than 10")
elif x == 10:
    print("x is equal to 10")
else:
    print("x is less than 10")
```

2. Loops: while and for

Loops allow a program to repeat a block of code multiple times, either for a specific number of iterations or until a condition is met.

2.1 while Loop:

A while loop repeats a block of code as long as a condition is True.

Syntax:

```
python
Copy code
while condition:
    # Code to execute while the condition is True
```

Example:

```
python
Copy code
i = 1
while i <= 5:
    print(i)
    i += 1
```

Explanation: This loop will print the numbers 1 through 5. The loop continues until i becomes greater than 5.

2.2 for Loop:

A for loop is used to iterate over a sequence (like a list, string, range, etc.).

Syntax:

```
python
Copy code
for variable in sequence:
    # Code to execute for each item in the sequence
```

Example:

```
python
Copy code
for i in range(1, 6):
    print(i)
```

Explanation: This loop will print the numbers 1 through 5, where the range(1, 6) function generates numbers from 1 to 5.

3. Nested Loops and Conditional Statements

3.1 Nested for Loops:

Nested loops are loops inside other loops. They allow for more complex iterations, such as printing patterns or working with multidimensional data.

Syntax:

```
python
Copy code
for variable1 in sequence1:
```

```
for variable2 in sequence2:  
    # Code to execute for each combination of items in sequence1 and sequence2
```

Example:

```
python  
Copy code  
for i in range(1, 4):  
    for j in range(1, 4):  
        print(f'i = {i}, j = {j}')
```

Explanation: This code will produce a combination of values of i and j, with the outer loop iterating over i and the inner loop iterating over j.

3.2 Nested if Statements:

Nested if statements are conditional statements placed inside other if statements, allowing for more specific conditions to be checked after an initial condition.

Syntax:

```
python  
Copy code  
if condition1:  
    if condition2:  
        # Code to execute if both condition1 and condition2 are True
```

Example:

```
python  
Copy code  
x = 5  
y = 10  
  
if x > 0:  
    if y > 5:  
        print("x is positive and y is greater than 5")
```

Explanation: The program checks both conditions: whether x is greater than 0 and whether y is greater than 5. If both conditions are true, the print statement is executed.

4. Break and Continue Statements

4.1 break Statement:

The break statement exits a loop prematurely when a condition is met.

Example:

```
python
Copy code
for i in range(1, 6):
    if i == 3:
        break
    print(i)
```

Explanation: The loop will print numbers 1 and 2, but when i becomes 3, the break statement is triggered, and the loop ends.

4.2 continue Statement:

The continue statement skips the current iteration of the loop and moves to the next iteration.

Example:

```
python
Copy code
for i in range(1, 6):
    if i == 3:
        continue
    print(i)
```

Explanation: The loop will print the numbers 1, 2, 4, and 5. When i is 3, the continue statement skips that iteration and moves to the next.

5. Use Cases for Control Structures

- **Decision-making:** Conditional statements are useful when different actions need to be taken based on input or specific conditions, like deciding whether a user qualifies for a loan or not.
- **Repetition:** Loops are commonly used for tasks that need to be repeated, such as processing every element in a list, generating sequences, or handling user input validation.
- **Nested logic:** Nested loops and conditionals are helpful when dealing with multi-dimensional data (such as matrices), building complex patterns, or handling scenarios where multiple layers of decisions are required.

6. Conclusion

Control structures such as if-else statements, while and for loops, and their nested forms are the foundation of flow control in programming. They allow developers to implement decision-making processes, repeat tasks, and handle complex logic efficiently. Understanding and mastering these control structures is key to writing efficient and effective code.

Exercise

1. Write a program that asks the user to enter a number. If the number is positive, print "Positive". If the number is negative, print "Negative". If the number is zero, print "Zero".
2. Write a program that takes a student's marks as input. Use nested if statements to assign a grade:
 - a $\geq 90 \rightarrow A$
 - b ≥ 75 and $< 90 \rightarrow B$
 - c ≥ 50 and $< 75 \rightarrow C$
 - d $< 50 \rightarrow F$
3. Write a program that prints the first 10 natural numbers using while statement.
4. Write a program that prints the squares of numbers from 1 to 10 using for loop
5. Write a program to print the following pattern using nested loops:

```

*
**
***
****
*****

```

6. Write a program which could generate the following pattern.

[illegible]

7. Write a program that generates a multiplication table (up to 10 x 10)
8. Write a program that calculates the sum of numbers from 1 to 100 using a while loop

Abstract:

This lab focuses on the implementation of conditional statements and loops in Python. The purpose of the lab was to understand and apply control structures such as `if`, `elif`, `else`, `while`, and `for` loops. We also practiced nested loops and nested conditional statements to solve different programming problems. The programs were written and executed using Visual Studio Code (VS Code). Through these exercises, we developed an understanding of decision-making, repetition, and pattern generation in Python.

Procedure:

- Opened **Visual Studio Code (VS Code)** and created a new Python file.
- Wrote Python programs for the given lab exercises.
- Used `if`, `elif`, and `else` statements to perform decision-making tasks.
- Used `while` loops to print natural numbers and calculate the sum from 1 to 100.
- Used `for` loops to calculate squares and generate multiplication tables.
- Used nested `for` loops to generate different star patterns.
- Used user input where required, such as entering a number or student's marks.
- Saved the Python file and executed the programs in VS Code.
- Observed the output and corrected any errors in the code.
- Verified that the output of each program matched the required exercise.

Conclusion:

In this lab, we successfully implemented different conditional statements and loops in Python using Visual Studio Code. The exercises helped us understand how programs make decisions using conditional statements and repeat tasks using loops. We also learned how nested loops can be used to generate patterns and perform repeated operations. Overall, the lab provided practical experience in using basic control structures for solving programming problems.

TASKS

Task 1:

```
1  num = float(input("Enter a number: "))
2
3  if num > 0:
4      print("Positive")
5  elif num < 0:
6      print("Negative")
7  else:
8      print("Zero")
```

Problems Output Debug Console Terminal Ports

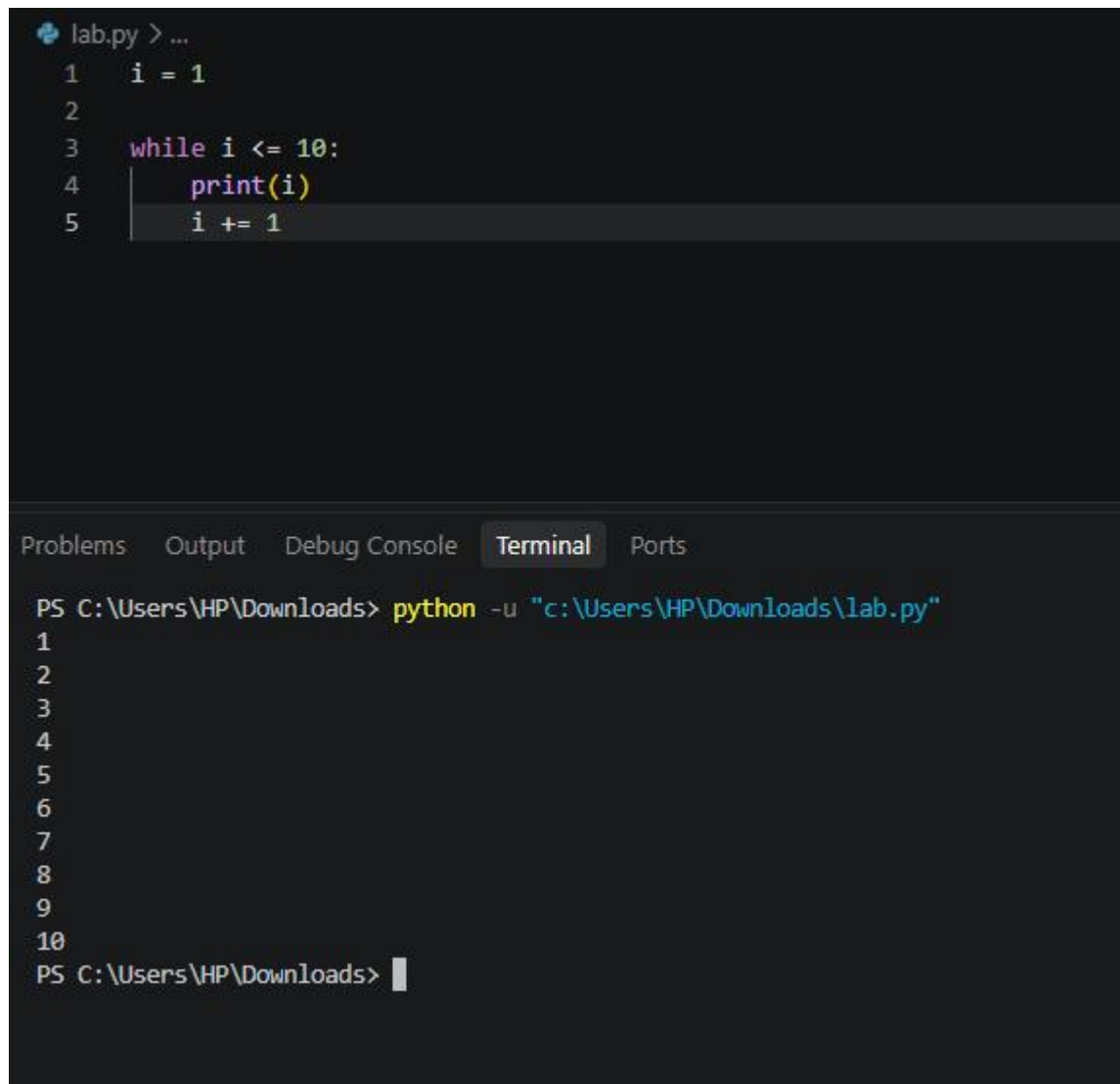
PS C:\Users\HP\Downloads> python -u "c:\Users\HP\Downloads\lab.py"
Enter a number: 50
Positive

Task 2:

```
lab.py > ...
1  marks = float(input("Enter student's marks: "))
2
3  if marks >= 90:
4      print("Grade A")
5  elif marks >= 75:
6      print("Grade B")
7  elif marks >= 50:
8      print("Grade C")
9  else:
10     print("Grade F")
```

Problems Output Debug Console Terminal Ports

PS C:\Users\HP\Downloads> python -u "c:\Users\HP\Downloads\lab.py"
Enter student's marks: 20
Grade F
PS C:\Users\HP\Downloads>

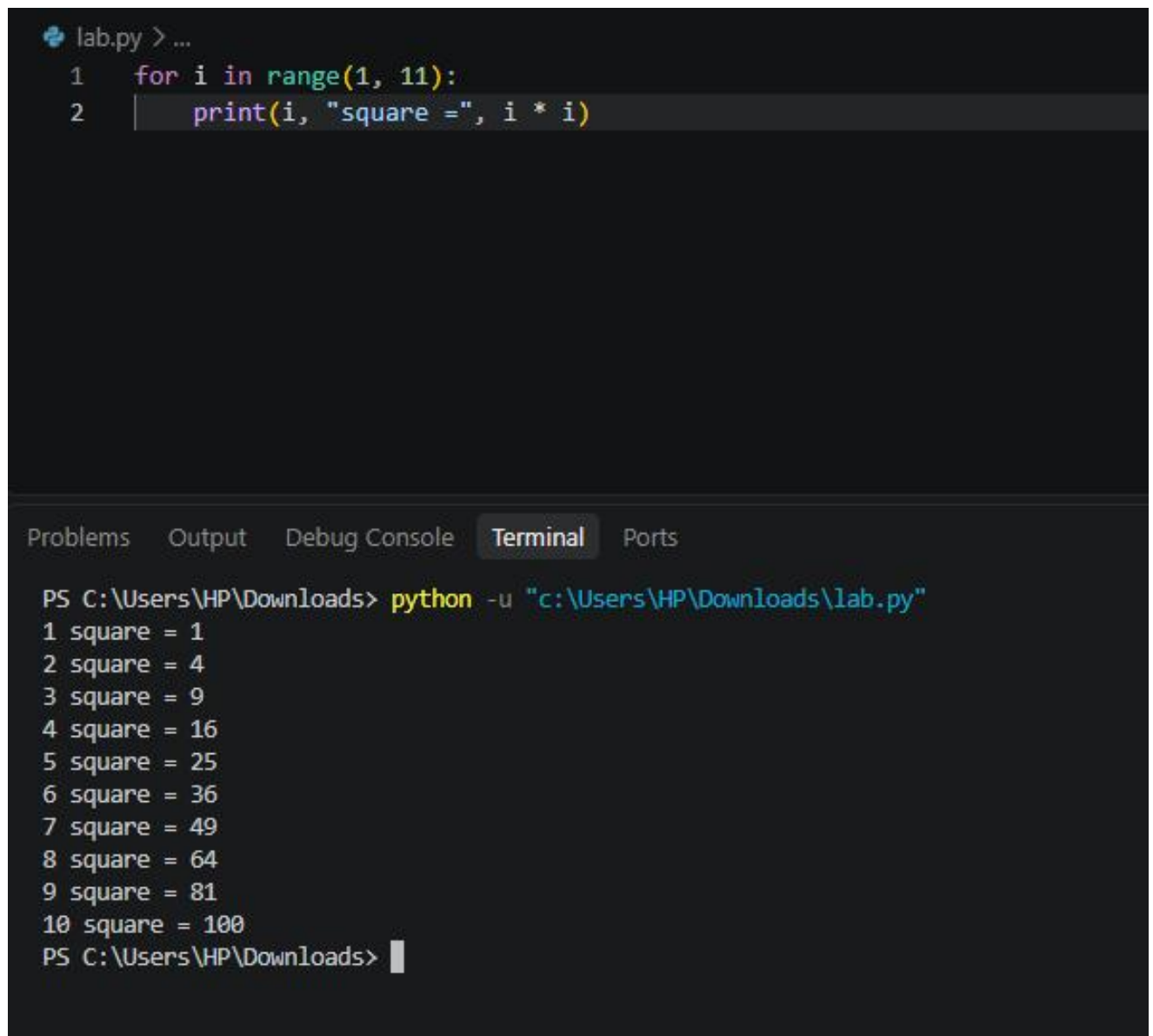
Task 3 :

The image shows a screenshot of a code editor and a terminal window. The code editor at the top displays a Python script named 'lab.py' with the following code:

```
lab.py > ...
1  i = 1
2
3  while i <= 10:
4      print(i)
5      i += 1
```

Below the code editor is a terminal window with tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active, showing the command to run the script and its output:

```
PS C:\Users\HP\Downloads> python -u "c:\Users\HP\Downloads\lab.py"
1
2
3
4
5
6
7
8
9
10
PS C:\Users\HP\Downloads> 
```

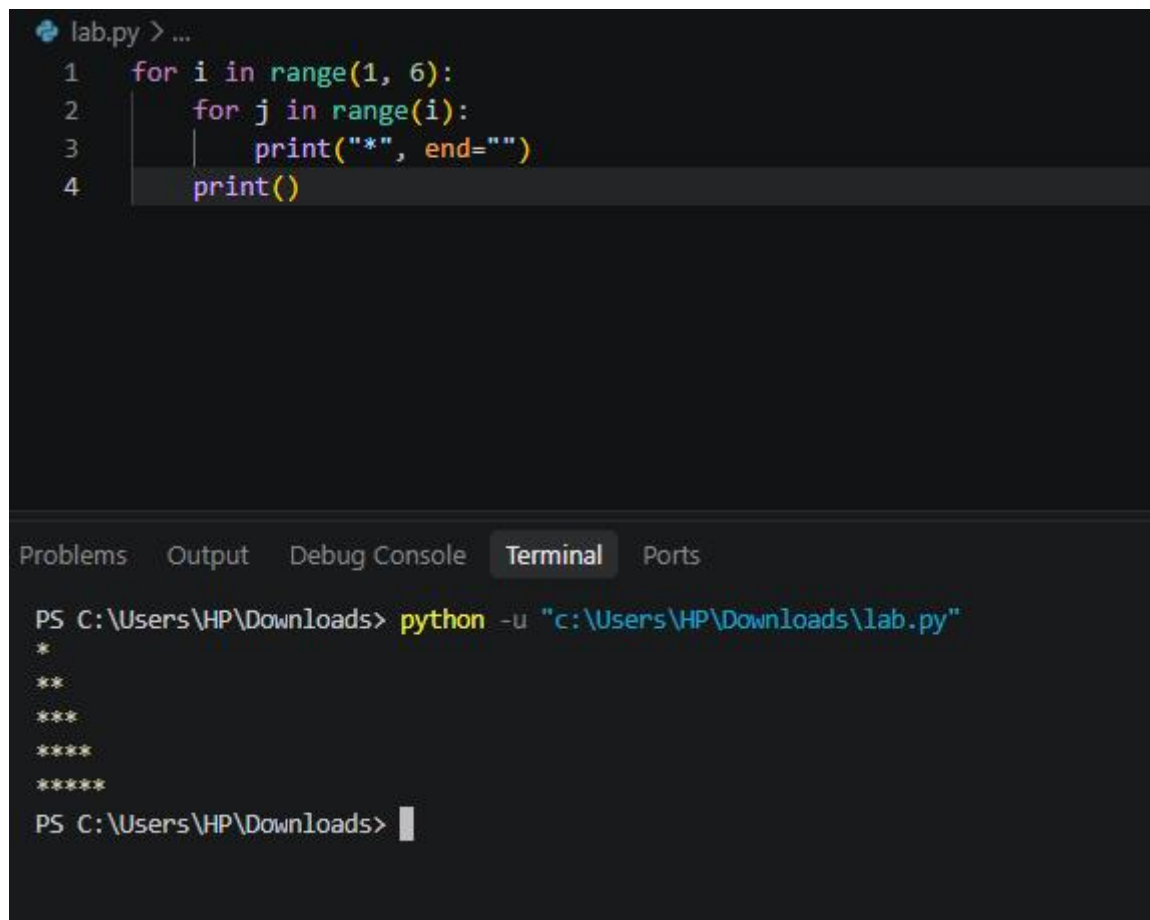
Task 4 :

The image shows a code editor window with a file named 'lab.py' containing two lines of Python code:

```
1 for i in range(1, 11):  
2     print(i, "square =", i * i)
```

Below the code editor is a terminal window with tabs for 'Problems', 'Output', 'Debug Console', 'Terminal', and 'Ports'. The 'Terminal' tab is active, showing the command to run the script and its output:

```
PS C:\Users\HP\Downloads> python -u "c:\Users\HP\Downloads\lab.py"  
1 square = 1  
2 square = 4  
3 square = 9  
4 square = 16  
5 square = 25  
6 square = 36  
7 square = 49  
8 square = 64  
9 square = 81  
10 square = 100  
PS C:\Users\HP\Downloads> |
```

Task 5 :

The image shows a code editor window with a file named 'lab.py' and a terminal window below it. The code in the editor is a Python script that prints a pattern of asterisks. The terminal shows the command to run the script and the resulting output.

```
lab.py > ...
1  for i in range(1, 6):
2      for j in range(i):
3          print("*", end="")
4      print()
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\HP\Downloads> python -u "c:\Users\HP\Downloads\lab.py"
*
**
***
****
*****
PS C:\Users\HP\Downloads> 
```

Task 6:

```
lab.py > ...
1  for i in range(1, 11):
2      for j in range(i):
3          print("*", end=" ")
4      print()
5
6  for i in range(9, 0, -1):
7      for j in range(i):
8          print("*", end=" ")
9      print()
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\HP\Downloads> python -u "c:\Users\HP\Downloads\lab.py"
*
* *
* * *
* * * *
* * * * *
* * * * * *
* * * * * * *
* * * * * * * *
* * * * * * * * *
* * * * * * * * * *
* * * * * * * * * * *
* * * * * * * * * * * *
* * * * * * * * * * * *
* * * * * * * * * * *
* * * * * * * * *
* * * * * * *
* * * * *
* * * *
* * *
* *
*
PS C:\Users\HP\Downloads> |
```

Task 7 :

```
lab.py > ...
1  for i in range(1, 11):
2      for j in range(1, 11):
3          print(i * j, end="\t")
4      print()
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\HP\Downloads> python -u "c:\Users\HP\Downloads\lab.py"
1      2      3      4      5      6      7      8      9      10
2      4      6      8      10     12     14     16     18     20
3      6      9      12     15     18     21     24     27     30
4      8      12     16     20     24     28     32     36     40
5      10     15     20     25     30     35     40     45     50
6      12     18     24     30     36     42     48     54     60
7      14     21     28     35     42     49     56     63     70
8      16     24     32     40     48     56     64     72     80
9      18     27     36     45     54     63     72     81     90
10     20     30     40     50     60     70     80     90     100
PS C:\Users\HP\Downloads> 
```

Task 8 :

```
lab.py > ...
2  total = 0
3
4  while i <= 100:
5      total += i
6      i += 1
7
8  print("Sum =", total)
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\HP\Downloads> python -u "c:\Users\HP\Downloads\lab.py"
Sum = 5050
PS C:\Users\HP\Downloads> 
```